

Unit 05

Transaction Processing Concepts

Ashwini Kumar Mathur [Assistant Professor]

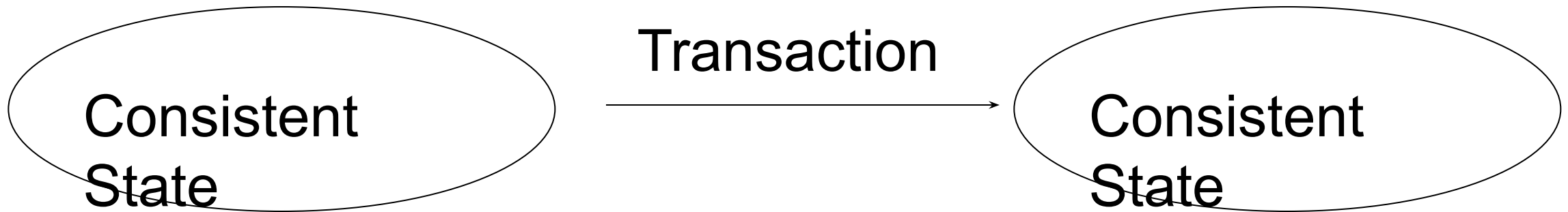
Outlines

1. Transaction
2. Acid Properties
3. Testing of Serializability
4. Serializability of Schedules
5. Conflict and view serializable schedule
6. Concurrency control

Transaction

- Programming abstraction
- Implement real-world transactions
 - Banking transaction
 - Airline reservation

Transaction: Programmer's Role



Transaction Concept

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
- **E.g., transaction to transfer \$50 from account A to account B:**
 1. **read**(A)
 2. $A := A - 50$
 3. **write**(A)
 4. **read**(B)
 5. $B := B + 50$
 6. **write**(B)
- **Two main issues to deal with:**
 - Failures of various kinds, such as hardware failures and system crashes
 - Concurrent execution of multiple transactions

Examples of Transactions [Fund Transfer]

- **Transaction to transfer \$50 from account A to account B:**

1. **read(A)**
2. $A := A - 50$
3. **write(A)**
4. **read(B)**
5. $B := B + 50$
6. **write(B)**

Examples of Transactions [Fund Transfer]

- Transaction to transfer \$50 from account A to account B:
 1. **read**(A)
 2. $A := A - 50$
 3. **write**(A)
 4. **read**(B)
 5. $B := B + 50$
 6. **write**(B)
- **Atomicity requirement**
 - If the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state
 - Failure could be due to software or hardware
 - **The system should ensure that updates of a partially executed transaction are not reflected in the database**
- **Durability requirement** — once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

Examples of Transactions [Fund Transfer] cont ..

- **Consistency requirement** in above example:
 - The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
 - Explicitly specified integrity constraints such as primary keys and foreign keys
 - Implicit integrity constraints
 - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand

Transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Examples of Transactions [Fund Transfer] cont ..

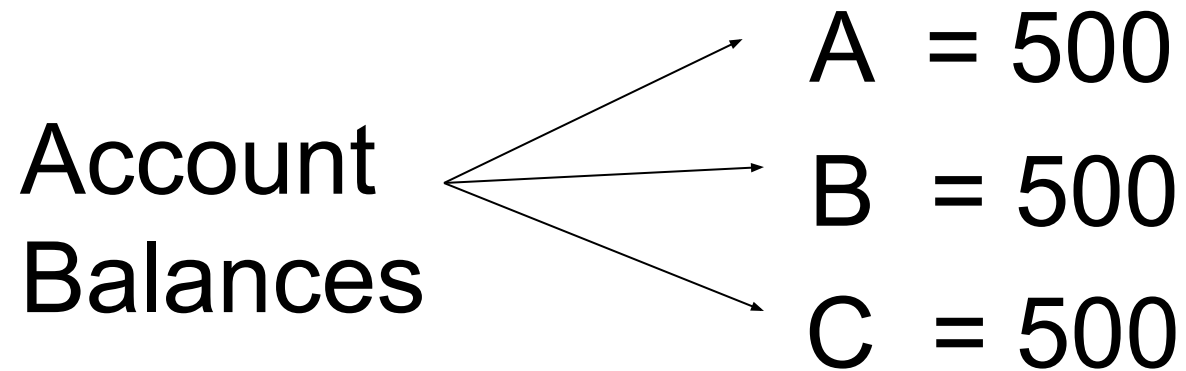
- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1	T2
1. read (A)	
2. $A := A - 50$	
3. write (A)	
	read(A), read(B), print(A+B)
4. read (B)	
5. $B := B + 50$	
6. write (B)	

- **Isolation can be ensured trivially by running transactions **serially**** (That is, one after the other.)

Issues with Concurrency: Example

Bank database: 3 Accounts



Property: $A + B + C = 1500$

Money does not leave the system

Transaction T1	Transaction T2	A	B	C
Read (A, t)		500	500	500
t = t - 100				
	Read (A, s)			
	s = s - 100			
	Write (A, s)	400	500	500
Write (A, t)		400	500	500
Read (B, t)				
t = t + 100				
Write (B, t)		400	600	500
	Read (C, s)			
	s = s + 100			
	Write (C, s)	400	600	600
		400 + 600 + 600 = 1600		

Transaction T1	Transaction T2	A	B	C
Read (A, t)		500	500	500
t = t - 100				
Write (A, t)		400	500	500
	Read (A, s)			
	s = s - 100			
	Write (A, s)	300	500	500
Read (B, t)				
t = t + 100				
Write (B, t)		300	600	500
	Read (C, s)			
	s = s + 100			
	Write (C, s)	300	600	600
		300 + 600 + 600 = 1500		

Exercise Question for Student Practices [04 mins]

Q1: Consider the following transaction T consisting of T1 and T2 : Transfer of 100 from account X to account Y. Explain the scenario of inconsistent database state.

Before: X : 500	Y: 200
Transaction T	
T1	T2
Read (X) $X := X - 100$ Write (X)	Read (Y) $Y := Y + 100$ Write (Y)
After: X : 400	Y : 300

ACID Properties [Summary]

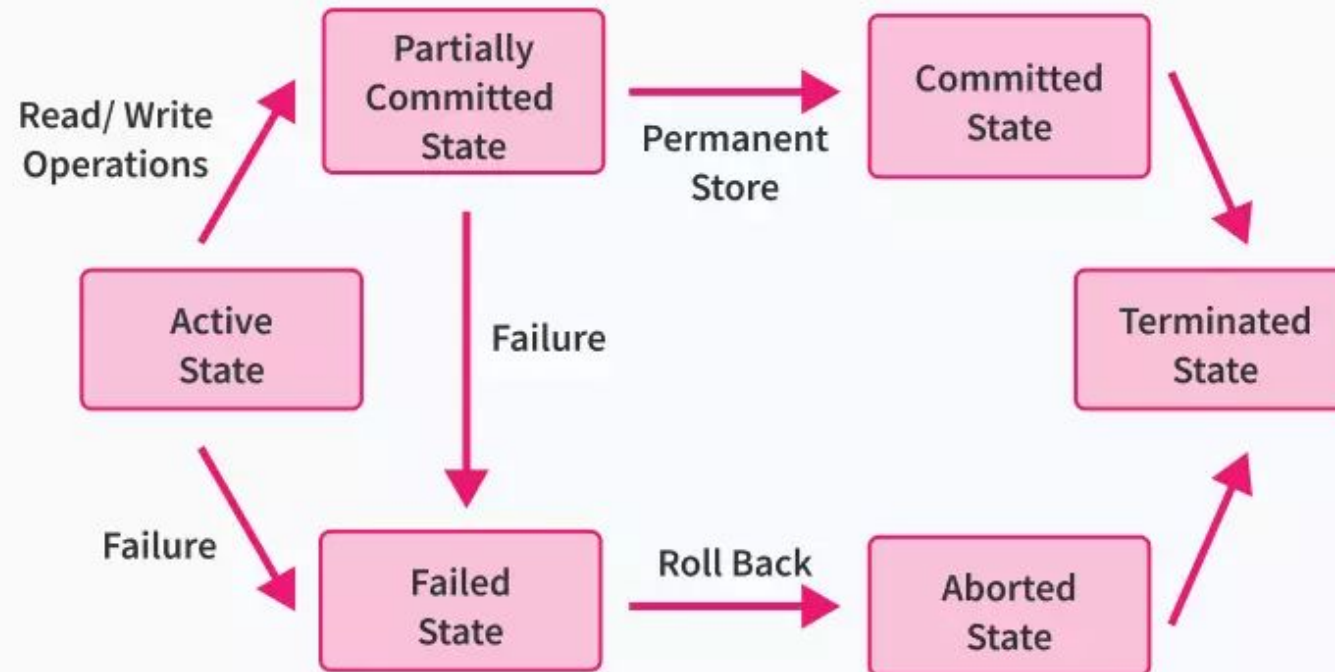
A **transaction** is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - Restart the transaction
 - Can be done only if no internal logical error
 - Kill the transaction
- **Committed** – after successful completion.

Transaction State .. continue



Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
 - **Increased processor and disk utilization**, leading to better transaction *throughput*
 - E.g., one transaction can be using the CPU while another is reading from or writing to the disk
 - **Reduced average response time** for transactions: short transactions need not wait behind long ones.
- **Concurrency control schemes** – mechanisms to achieve isolation
 - That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database

Schedules [Set of Transactions]

- **Schedule** – a **sequences of instructions** that specify the chronological order in which instructions of concurrent transactions are executed
 - A schedule for a **set of transactions** must consist of all instructions of those transactions
 - **Must preserve the order in which the instructions appear in each individual transaction.**
- A transaction that successfully completes its execution will have a commit instructions as the last statement
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

Schedule [01] ..

- Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B .
- A **serial** schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Schedule 1

- Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B
- An example of a **serial** schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

A	B	A+B	Transaction	Remarks
100	200	300	@ Start	
50	200	250	T1, write A	
50	250	300	T1, write B	@ Commit
45	250	295	T2, write A	
45	255	300	T2, write B	@Commit

Consistent @ Commit

Inconsistent @ Transit

Inconsistent @ Commit

Schedule [02] ..

- A serial schedule where T_2 is followed by T_1

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Schedule 2

- A **serial** schedule in which T_2 is followed by T_1 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

A	B	A+B	Transaction	Remarks
100	200	300	@ Start	
90	200	290	T2, write A	
90	210	300	T2, write B	@ Commit
40	210	250	T1, write A	
40	260	300	T1, write B	@Commit

Consistent @ Commit

Inconsistent @ Transit

Inconsistent @ Commit

Values of A & B are different from
Schedule 1 – yet consistent

Schedule [03] ..

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1

T_1	T_2
read (A) $A := A - 50$ write (A)	
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	
	read (B) $B := B + temp$ write (B) commit

- In Schedules 1, 2 and 3, the sum $A + B$ is preserved.

Schedule 3

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is **equivalent** to Schedule 1

T_1	T_2	T_1	T_2	A	B	A+B	Transaction	Remarks
read (A) $A := A - 50$ write (A)		read (A) $A := A - 50$ write (A)		100	200	300	@ Start	
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)	read (B) $B := B + 50$ write (B) commit		50	200	250	T1, write A	
				45	200	245	T2, write A	
read (B) $B := B + 50$ write (B) commit			read (A) $temp := A * 0.1$ $A := A - temp$ write (A)	45	250	295	T1, write B	@ Commit
	read (B) $B := B + temp$ write (B) commit		read (B) $B := B + temp$ write (B) commit	45	255	300	T2, write B	@Commit

Schedule 3

Schedule 1

Consistent @ Commit

Inconsistent @ Transit

Inconsistent @ Commit

Schedule [04] ..

- The following concurrent schedule does not preserve the value of $(A + B)$.

T_1	T_2
read (A) $A := A - 50$	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B)
write (A) read (B) $B := B + 50$ write (B) commit	$B := B + temp$ write (B) commit



Serializability :

- What does it mean for transactions to “appear” as if they were run in sequence?

Serializability ..

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
 1. **Conflict serializability**
 2. **View serializability**

Simplified View of Transaction

- We ignore operations other than **read** and **write** instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only **read** and **write** instructions.

Conflicting Instructions

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j , and at least one of these instructions wrote Q .
 - $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
 - $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They conflict.
 - $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They conflict.
 - $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They conflict.
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
- If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

Conflict Serializability

- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.
- We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule

Conflict Serializable Schedule

- A schedule is called conflict serializability if **after swapping of non-conflicting operations, it can transform into a serial schedule.**
- The schedule will be a conflict serializable if it is conflict equivalent to a serial schedule.

Conflicting Operations

The two operations become conflicting if all conditions satisfy:

1. Both belong to separate transactions.
2. They have the same data item.
3. They contain at least one write operation.

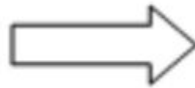
Example:

Swapping is possible only if S1 and S2 are logically equal.

1. T1: Read(A) T2: Read(A)

T1	T2
Read(A)	
	Read(A)

Swapped



T1	T2
	Read(A)
Read(A)	

Schedule S1

Schedule S2

Here, S1 = S2. That means it is non-conflict.

2. T₁: Read(A) T₂: Write(A)

T ₁	T ₂
Read(A)	Write(A)

Swapped



T ₁	T ₂
Read(A)	Write(A)

Schedule S₁

Schedule S₂

Here, $S_1 \neq S_2$. That means it is conflict.

Conflict Equivalent

In the conflict equivalent, one can be transformed to another by swapping non-conflicting operations. In the given example, S2 is conflict equivalent to S1 (S1 can be converted to S2 by swapping non-conflicting operations).

Two schedules are said to be conflict equivalent if and only if:

1. They contain the same set of the transaction.
2. If each pair of conflict operations are ordered in the same way.

Conflict Serializability continue..

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

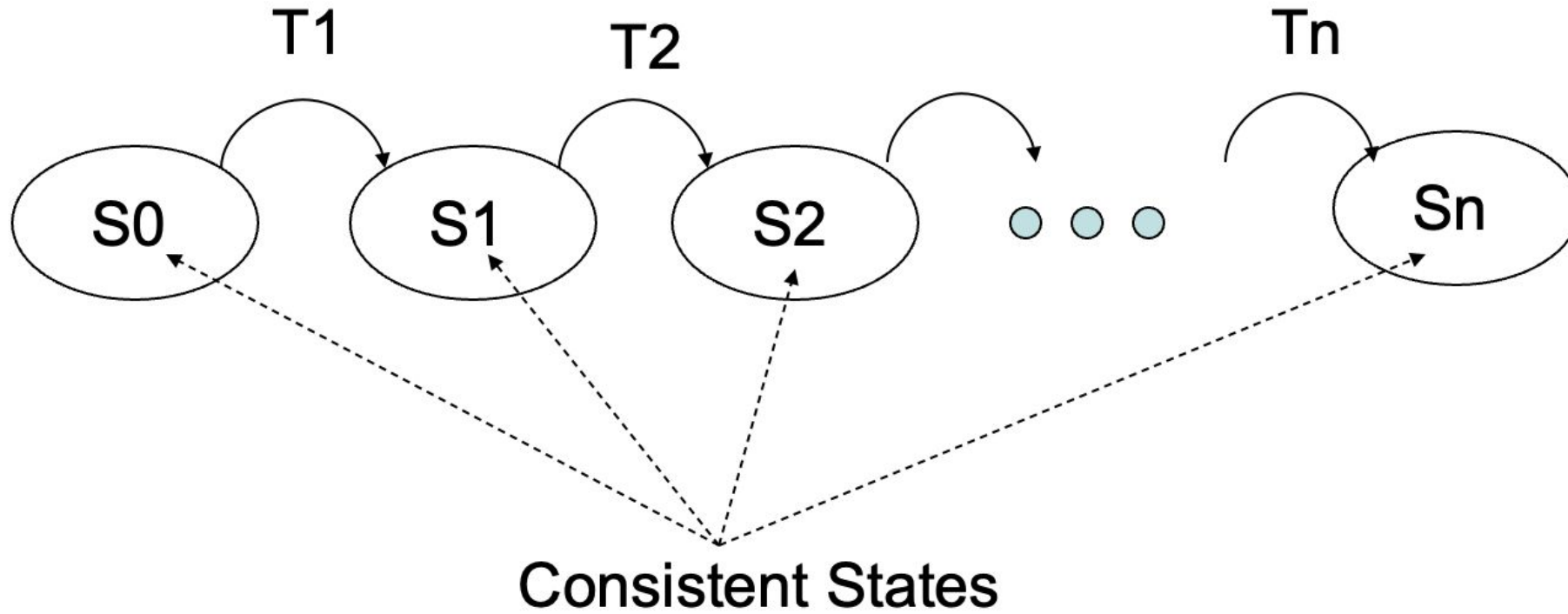
T_1	T_2
read (A) write (A)	read (A) write (A)
read (B) write (B)	read (B) write (B)

Schedule 3

T_1	T_2
read (A) write (A) read (B) write (B)	read (A) write (A) read (B) write (B)

Schedule 6

Serial Schedule



Is this Serializable?

Read (A, t)

$t = t - 100$

Write (A, t)



Read (B, t)

$t = t + 100$

Write (B, t)

Read (A, s)

$s = s - 100$

Write (A, s)



Read (C, s)

$s = s + 100$

Write (C, s)

Transaction T1

Transaction T2

Equivalent Serial Schedule

Read (A, t)

$t = t - 100$

Write (A, t)

Read (B, t)

$t = t + 100$

Write (B, t)

Read (A, s)

$s = s - 100$

Write (A, s)

Read (C, s)

$s = s + 100$

Write (C, s)

Transaction T1

Transaction T2

Is this Serializable?

Read (A, t)
 $t = t - 100$

Write (A, t)
Read (B, t)
 $t = t + 100$
Write (B, t)

Transaction T1

Read (A, s)
 $s = s - 100$
Write (A, s)

Read (C, s)
 $s = s + 100$
Write (C, s)

Transaction T2

No. In fact, it leads
to inconsistent state

Conflict Serializability continue..

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.

View Serializability

- A schedule will view serializable if it is view equivalent to a serial schedule.
- If a schedule is conflict serializable, then it will be view serializable.
- The view serializable which does not conflict serializable contains blind writes.

View Equivalent

Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions:

1. Initial Read

An initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

T1	T2
Read(A)	Write(A)

Schedule S1

T1	T2
Read(A)	Write(A)

Schedule S2

Updated Read

In schedule S1, if T_i is reading A which is updated by T_j then in S2 also, T_i should read A which is updated by T_j .

T1	T2	T3
Write(A)	Write(A)	Read(A)

Schedule S1

T1	T2	T3
Write(A)	Write(A)	<u>Read(A)</u>

Schedule S2

Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

3. Final Write

A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S1

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S2

Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

Example:

T1	T2	T3
Read(A) Write(A)	Write(A)	 Write(A)

Schedule S

With 3 transactions, the total number of possible schedule

Given the following transactions and their operations:

- **T1:** Read(A), Write(A)
- **T2:** Write(A)
- **T3:** Write(A)

How many possible schedules can be formed with these three transactions?

Solution:

The number of possible schedules can be calculated as the factorial of the number of transactions. In this case, we have 3 transactions, so the number of possible schedules is $3! = 3 * 2 * 1 = 6$.

Is this schedule conflict-serializable?

Solution:

Yes, this schedule is conflict-serializable. A schedule is conflict-serializable if there exists a serial schedule that is equivalent to it. In this case, the serial schedule (T1, T2, T3) is equivalent to the given schedule S.

Explanation:

In both schedules, T1 reads and writes A before T2 and T3 write A. This ensures that the final value of A is consistent, regardless of the order of execution.

View Serializability

- Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for each data item Q ,
 1. If in schedule S , transaction T_i reads the initial value of Q , then in schedule S' also transaction T_i must read the initial value of Q .
 2. If in schedule S transaction T_i executes **read**(Q), and that value was produced by transaction T_j (if any), then in schedule S' also transaction T_i must read the value of Q that was produced by the same **write**(Q) operation of transaction T_j .
 3. The transaction (if any) that performs the final **write**(Q) operation in schedule S must also perform the final **write**(Q) operation in schedule S' .
- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.

View Serializability continue..

- A schedule S is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		write (Q)

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has **blind writes**.

Concurrency Control

- A database must provide a mechanism that will ensure that all possible schedules are
 - either conflict or view serializable, and
 - are recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
 - Are serial schedules recoverable/cascadeless?
- Testing a schedule for serializability *after* it has executed is a little too late!
- **Goal** – to develop concurrency control protocols that will assure serializability.

Concurrency Control continue ..

- Schedules must be conflict or view serializable, and recoverable, for the sake of database consistency, and preferably cascadeless.
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency.
- Concurrency-control schemes tradeoff between the amount of concurrency they allow and the amount of overhead that they incur.
- Some schemes allow only conflict-serializable schedules to be generated, while others allow view-serializable schedules that are not conflict-serializable.

Concurrency Control VS Serializability Tests

- Concurrency-control protocols allow concurrent schedules, but ensure that the schedules are conflict/view serializable, and are recoverable and cascadeless .
- Concurrency control protocols (generally) do not examine the precedence graph as it is being created
 - Instead a protocol imposes a discipline that avoids non-serializable schedules.
 - We study such protocols in Chapter 16.
- Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.
- Tests for serializability help us understand why a concurrency control protocol is correct.

Weak Level of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
 - E.g., a read-only transaction that wants to get an approximate total balance of all accounts
 - E.g., database statistics computed for query optimization can be approximate (why?)
 - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance